# Complex Networks and Intrusion Detection Systems with Artifical Intelligence (AI)

Jared Wood

March 18, 2026


## Abstract

This paper explores the intersection of complex network theory and cybersecurity through the development of an AI driven Intrusion Detection System (IDS). By leveraging topological features derived from network graph structures, this research aims to identify anomalous patterns that traditional tabular-based systems frequently overlook. Using the NF-UQ-NIDS-v2 dataset as a baseline, a comparative study was conducted between two distinct feature sets: a standard "tabular" dataset and a "complex" dataset composed of topological metrics. Features were engineered using industry-standard libraries including iGraph, NumPy, and SciPy.

Results indicate that AI models trained on complex network data show a drastic improvement in identifying "stealthy" cybersecurity attacks, such as Worms and Theft, which often bypass volume-based detectors. Conversely, a performance trade-off was observed in the detection of "obvious" volumetric attacks, such as DoS and DDoS. While currently a proof-of-concept, this research demonstrates that coupling topological pattern recognition with machine learning provides a significant performance uplift in cybersecurity and potentially offers transformative applications in medical informatics, intelligence gathering, and e-commerce.


## Introduction

Over the last few years, AI has made tremendous improvements from the notorious and funny "Will Smith eating spaghetti" video to passing numerous higher-level tests on aspects in the STEM-field. Arguably, most applications of AI largely have to do with ingesting large data to expedite decision-making and understanding of an issue, especially for high-stakes environments like the military. Unfortunately, the application of AI largely relies on the concept of "backpropagation" where in general, a model receives input information, makes numerous calculus, statistical, and probabilistic calculations to determine an answer, and if the answer is wrong, it back-propagates through every decision and adjusts the weights and biases to better bias the model towards the correct answer. In short, it performs a very high-level version of brute forcing the correct answer. For this reason, whenever an AI provides the wrong answer - or really whenever someone wants to understand how the answer is being derived - it

becomes very difficult to explain the AI's decision making process which leads to the general notion of the "black box", or unexplainable AI.

In contrast to this, there's a large push for explainable AI which leads to much better information gain in a team since it should be easier to isolate what matters and what doesn't matter for training. In support of explainable AI, complex network theory provides a multitude of describing and deriving topological patterns so as long as the data can be represented as a graph, which relies heavily on matrix-friendly data. In addition to supporting explainable AI, the nature of complex network theory should significantly improve the performance of an AI model by allowing to easily delineate between anomalous entries and a more robust model that is significantly less susceptible to data and model poisoning since much of complex network information is all tightly woven together.

In order to test and observe this idea, the "NF-UQ-NIDS-v2 Network Intrusion Detection Dataset" by Arya Shah on Kaggle will be used and sent through a specific pre-processing and model training pipeline to isolate the difference in model performance largely through a non-complex network dataset, named the "Tabular Dataset", and a complex network dataset, named the "Complex Dataset".

## Methodology

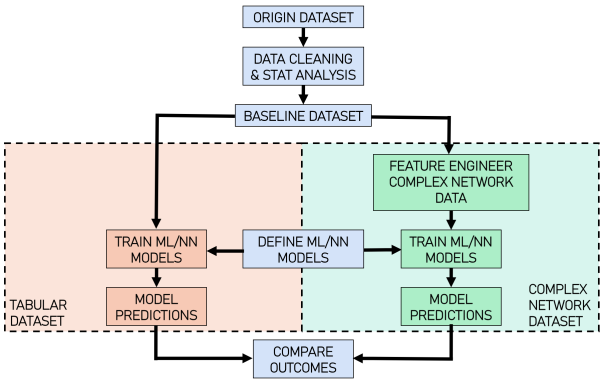


Figure 1: Experimental Pipeline - Comparative Methodology for Tabular vs. Complex Datasets and AI model training

**Origin Dataset:** The "origin dataset" comes from the "NF-UQ-NIDS-v2 Network Intrusion Detection Dataset" by Arya Shah on Kaggle. The dataset consists of 75,987,976 rows and 46 columns. From my understanding, this dataset houses information that is generally expected for cybersecurity purposes. The target classification column is the 'Attack' column which consists of 21 categories. One such category is "Benign" denoting connections where no malicious activity was conducted and the other 20 categories are various types of malicious activity (i.e. "DDoS", "Worms", "Randomware", etc.)

**Data Cleaning and Statistical Analysis:** Initial dataset cleaning was conducted and removed duplicate entries, and any null and problematic entries. Problematic entries are generally checked for data in columns that don't make sense (i.e. categorical data, but a numerical value exists). No nulls were found, but 0.0.0.0 IPs were found which were initially kept since I don't know if this information had anything useful. After dataset cleaning, the dataset was reduced to 5% of the dataset size and was stratified across the "Attack" column for general expediency of data exploration. Stratification just ensures that the distribution of all the different attack types are maintained.

Due to lack of specific domain knowledge in cybersecurity and general time constraints, statistical analysis and associative analysis of every column was conducted to further determine what information is significant for model training and what is considered "noisy" and would negatively affect the AI model performance. Generally, I wanted to not only determine that the information is not random via statistical testing as well as if the information has any sort of association with the target column. For this reason, every column was determined to either be of a categorical or continuous based information. For categorical columns, chi-squared-contingency was used as statistical testing and Kramer's V was used for associativity analysis. For continuous columns, Kruskal-Wallis was used for statistical testing and Mutual In-

formation was used for associativity analysis.

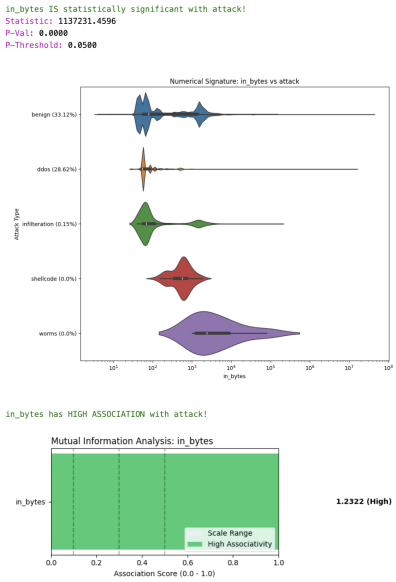


Figure 2: Continuous column statistical and associative analysis example

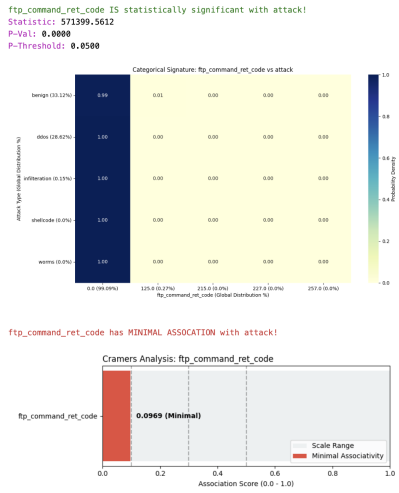


Figure 3: Categorical column statistical and associative analysis example

**Baseline/Tabular Dataset:** After cleaning and statistical analysis, the remaining dataset was 3,799,095 rows and 13 training columns and all columns were normalized and encoded where necessary to prevent biasing towards larger ranges and smaller ranges in continuous columns (Excludes the target columns). This dataset is considered the "Tabular Dataset" and is ready for the respective tabular model training. The distribution is slightly imbalanced where $\approx \frac{1}{3}$ of the dataset is non-malicious connections and $\approx \frac{2}{3}$ of the dataset is malicious connections. The distribution of malicious attacks are very imbalanced especially between the more "obvious" attacks like DDoS and the more "stealthy" attacks like Worms. Generally for the sake of AI training, this imbalance should lead to significant issues in predictions of "stealthy" attacks.

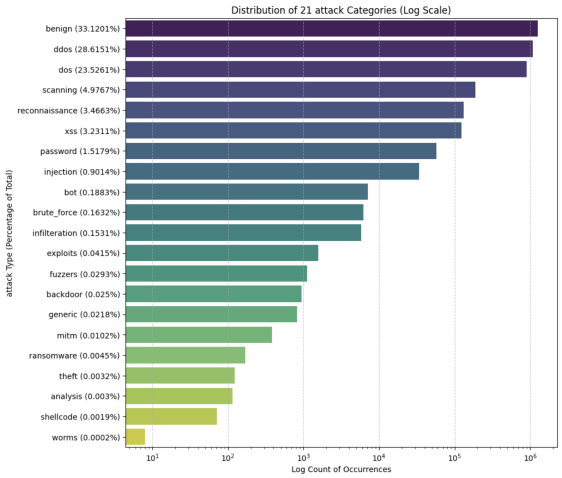


Figure 4: Log-Scale Target Distribution of Baseline Dataset/Tabular Dataset

**Tabular vs. Complex Dataset:** Complex Network theory was applied to the "Tabular Dataset" in order to create a "Complex Dataset" that is as close as possible in terms of what is shown from the dataset. In theory, this was done as another layer of isolating the independent variable. These datasets will then be sent into separate, but similar structured ML and NN models to see if the different datasets on the same information had any significant outcomes. Important to note, the Complex Dataset became very imbalanced where over 99% of entries were non-malicious and less than 1% of entries were malicious connections. This is significant because this imbalance likely leads to an overfitted model that can only detects non-malicious connections.

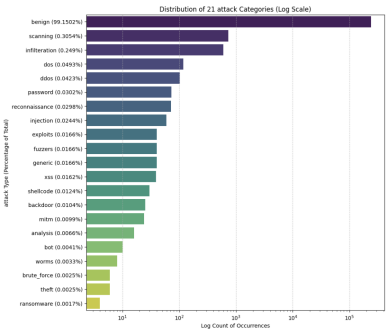


Figure 5: Log-Scale Target Distribution of Complex Dataset

**Complex Network Feature Engineering:** Primarily because of time constraints, python libraries iGraph, scipy, and numpy were very helpful in expediting the acquisition of complex network information since they offered already built functions for things like eigenvalues, eigenvectors, pagerank, etc. The largest issue was that the "Tabular Dataset" was not matrix friendly because there were multiple cases of the same source IP connecting to the same destination IP in the worst case, 190,117 times.

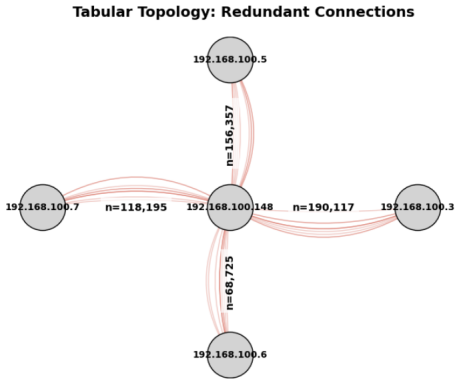


Figure 6: Example of repetitive connections

For this reason, the dataset was grouped first by source IP, destination IP, then attack type. During this grouping, in bytes, out bytes, and duration in were aggregated to maintain the overall importance of the connection between the two nodes. This significantly reduced the amount of entries due to grouping, but made the dataset closer to being matrix friendly.

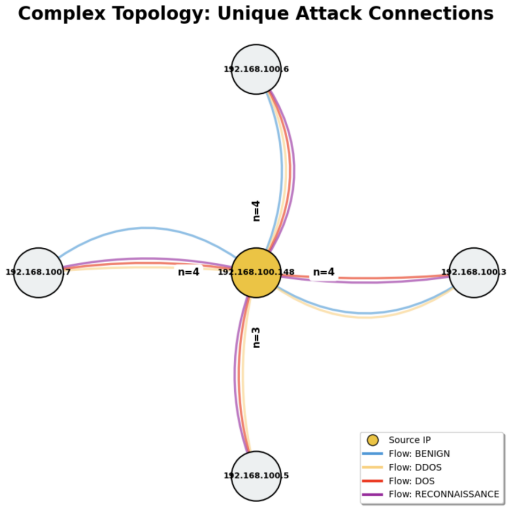


Figure 7: Example of grouped connections

The other issue was that the overall dataset was not a connected graph. To remedy this, graph IDs were created for every connection in the dataset to associate what connection belonged to what connected graph.

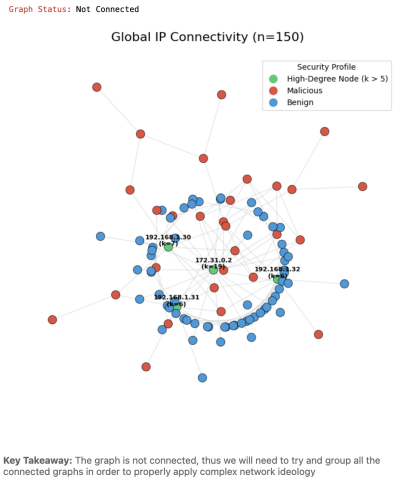


Figure 8: Sample of the disconnected graph from the dataset

At this point, complex network theory information derived were the first and second eigenvalues to denote the spectral gap, the first and second eigenvectors, source IP and destination IP page ranks, global and node-based steps to convergence and the edge-weight as means to hopefully maintain the importance between ev-

ery connection. This ended in the "Complex Dataset" with 241,361 rows and 16 columns for training (Excludes the target columns and the IPs to avoid target leakage). To emphasize from earlier, the "Complex Dataset" is very imbalanced with over 99% of data as non-malicious connections and less than 1% of data as malicious connections which was initially worrisome because this should likely lead to an extremely over-fitted model that can only detect non-malicious connections properly. At the end of this, we had 4 connected graphs that are weighted and directed, the IPs represent the individual nodes within the graph, and the attack-type relates to the edge between two IPs. All engineered information that was to be specific to individual nodes were tied to the source IP for every entry. There's much that could've been explored in more depth to ensure information was not redundant or incorrectly embedded into the dataset, but there wasn't much time to do so.

**ML and NN Model Definition:** The purpose of having a ML and NN model is to observe whether backpropagation (Heavy application of calculus, statistics, and probability) allows for more performance as opposed to the selected ML model. In theory, the NN should always perform better, but ideally, we want to see if we can get the ML model to perform just as good or better since ML models are easier to explain. The ML model selected was a Random Forest Classifier to help reduce the issue of massive imbalance in the "Complex Dataset" and the NN was just a simple forward-pass tensorflow/keras model that was constructed to mimic the Random Forest Classifier as much as possible. This went as far as having an input layer, 3 hidden layers, and an output layer. This was intentionally left as simple as possible again to isolate the independent variable in this paper.

**Tabular/Complex AI Training and Predictions:** This was fairly simple where a ML and NN model was trained on the "Tabular Dataset" and a similar structured ML and NN model was trained on the "Complex Dataset". Once trained, they simply made predictions on their respective testing datasets which was just 10% of their respective datasets that the model never gets trained on.

**Evaluation:** This is as simple as just observing the metrics produced from the respective Tabular/Complex trained models and the ML/NN models. The most important metrics for evaluation are precision and recall. Precision denotes how well the model correctly flags what a connection's attack type is. Recall denotes how well the model flags a connection's attack type that isn't related to the attack type for observation. In other words, if we assume our attack type we are focusing on is "Bengin", precision denotes how many connections that were "Benign" were accurately predicted as "Benign" and recall denotes how many connections that were not "Benign" were accurately predicted as anything else that was not "Benign".

## Experimental Results

The Tabular/Complex trained NN models performed worse than their respective ML counterparts likely as a combination of the dataset imbalance and heavy statistical analysis ahead of time before model training. Between the ML trained Tabular/Complex model, the Tabular trained model generally performed better in predicting "obvious" malicious attacks and the Complex trained model performed significantly better in predicting "stealthy" malicious attacks.

## Discussion

Before anything else, this project was conducted in very general applications of complex network theory as well as quick decisions due to time constraints. For this reason, there's a multitude of areas that need further exploration and analysis as well experimentation to better understand why improvements were observed. Besides this, interestingly, both NN models performed worse than their respective ML models.

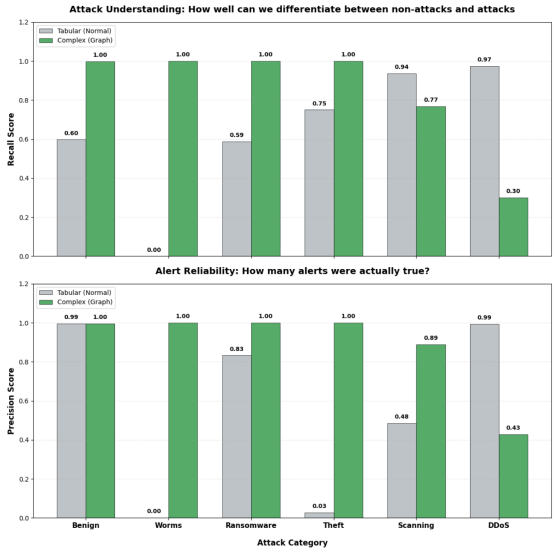


Figure 9: Precision/Recall performance sample of Tabular/Complex ML trained models

This may be due to the NN structure being too simple or may hint towards that giving a NN model patterns to train off of via complex network theory is largely ignored and will force its own definition of patterns and outcomes. In relation to the ML models, generally the Tabular trained ML model had a better time identifying "obvious" attacks like DDoS whereas the Complex trained ML model has a significant improvement in identifying "stealthy" attacks. This finding is even more significant given that not only was the distribution of malicious attacks less than 1% of the dataset, but the "stealthy" attacks made up less than half of that percent. This denotes that even though there was nearly no information for the model to train off of for "stealthy" attacks, application of complex network theory made it very easy to identify "stealthy" attacks likely following the notion that "stealthy" attacks tend to follow a topological pattern; On this note, the Tabular trained dataset may have issues identifying "stealthy" attacks since they tend to mimic non-malicious attacks which makes basic "Rate-of-Change" math application. Outside of this, the Complex trained ML model appeared to have worse performance for "obvious" attacks likely because of over-fitting towards underlying topological patterns of the overall connections network. This is also important to observe because it's a significant avenue of future exploration where my method of grouping and aggregation of edge-weights likely destroyed this information making it hard to derive for the Complex trained ML model. Alternatively, if this isn't the case, it may denote where complex network theory fails.

| | ipv4_src_addr | ipv4_dst_addr | in_bytes | out_bytes | attack |
|---|---|---|---|---|---|
| 53864551 | 192.168.1.31 | 192.168.35.45 | 44 | 0 | scanning |
| 1117388 | 192.168.1.34 | 192.168.1.49 | 44 | 0 | benign |
| 58230445 | 192.168.100.149 | 192.168.100.5 | 44 | 40 | reconnaissance |
| 27953837 | 172.31.69.13 | 172.31.69.7 | 44 | 40 | infilteration |
| 43132694 | 192.168.100.3 | 192.168.100.148 | 44 | 180 | ddos |
| 39032043 | 192.168.1.31 | 192.168.1.195 | 44 | 44 | password |
| 10984851 | 192.168.100.7 | 192.168.100.148 | 44 | 180 | dos |

Figure 10: Example of why Tabular trained models may fail

## Conclusion

Overall, this project at the very least shows that complex network theory may have a very significant application not just with AI and intrusion detection systems, but because of the topological patterns it describes, the wide potential of applications throughout any field! So as long as you can demonstrate the data effectively as a connected graph which allows the information to be manipulated through matrices. For this particular use-case, complex network theory made "stealthy" connections very easy to identify even with the dataset being over 99% non-malicious connections. To reiterate, this project was very general in the application of complex network theory and decisions made were fairly rushed and "path of least resistance" to an extent due to time constraints. For this reason, there's likely many things that may have been coded incorrectly, derived incorrectly, or caused data leakage/poisoning during model training and testing. There's a multitude of aspects that could be improved and explored in further depth from this project and there's so much potential that Complex Network theory can offer in so many fields!

## Resources Used

- **Hardware:** Macbook Laptop
- **Software:** Overleaf, VSCode, Python
- **Resources:** MATH3605 Course Notes, Kaggle Dataset Embedded Link